



# Traceability Management for Impact Analysis

*Andrea De Lucia, Fausto Fasano, Rocco Oliveto*

Software Engineering Lab  
Department of Mathematics and Informatics  
University of Salerno  
84084, Fisciano (SA), Italy  
*adelucia@unisa.it*

Frontiers of Software Maintenance – Beijing, 30 September 2008

1

## Outline

- Introduction
- Impact Analysis
- Traceability Dimensions
- Traceability Challenges
- Conclusion



Frontiers of Software Maintenance – Beijing, 30 September 2008

2

## Outline

- Introduction
- Impact Analysis
- Traceability Dimensions
- Traceability Challenges
- Conclusion

Frontiers of Software Maintenance – Beijing, 30 September 2008

3

## Software Maintenance and Impact Analysis

- Software maintenance
  - The process of modifying a software system or component after delivery to correct faults, improve performance or other attributes, or adapt to a changed environment
- Impact analysis
  - Analysing parts of the system that will be affected by the change and examine them for possible further impacts
  - Looking for ripple-effects ...
- A change has not only impact on the source code, but also on the other related software artefacts, such as requirement, design, and test artefacts

Frontiers of Software Maintenance – Beijing, 30 September 2008

4

## Impact Analysis and Traceability Management

- Impact analysis can be efficiently supported through traceability information.
  - Traceability: *“the ability to describe and follow the life of an artefact, in both a forwards and backwards direction”*
  - Traceability links help software engineers to understand the relationships and dependencies among various software artefacts.
  - Once a maintainer has identified the high-level document (e.g., requirement, use case) related to the feature to be changed, traceability helps to locate the pieces of design, code and whatever need to be maintained.

Frontiers of Software Maintenance – Beijing, 30 September 2008

5

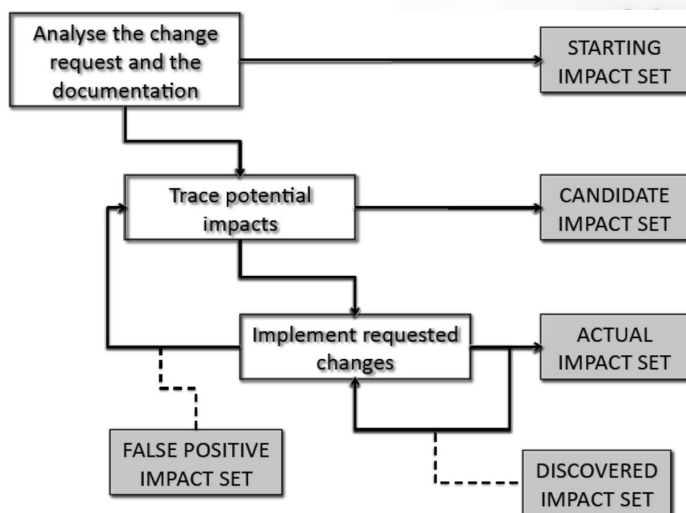
## Outline

- Introduction
- Impact Analysis
- Traceability Dimensions
- Traceability Challenges
- Conclusion

Frontiers of Software Maintenance – Beijing, 30 September 2008

6

## Impact Analysis Process



Adapted from:  
Bohner (ICSM 2002)

Frontiers of Software Maintenance – Beijing, 30 September 2008

7

## Goal of Impact Analysis Process

- Estimating a CIS that is as close as possible to the AIS
- The accuracy of the impact analysis process can be evaluated using several metrics. Examples:
  - **Recall** measures the percentage of actual impacts included in CIS:  $|CIS \cap AIS|$  divided by  $|AIS|$ 
    - Recall is 1 when DIS is empty
  - **Precision** measures the percentage of candidate impacts that are actual impacts:  $|CIS \cap AIS|$  divided by  $|CIS|$ 
    - Precision is 1 when FPIS is empty
  - If  $CIS = AIS$ , both recall and precision are 1
    - Unfortunately, this happens rarely, so there is the need for a trade-off

Frontiers of Software Maintenance – Beijing, 30 September 2008

8

## Identifying the Starting Impact Set

- The larger the system the higher the effort required to identify the SIS
- Mapping the concepts defined in a change request specification onto source code components is a difficult task
  - Concept assignment: “discovering human-oriented concepts and assigning them to their realizations”
  - This process cannot be completely automated, as concepts and programs are not at the same level of abstraction

Frontiers of Software Maintenance – Beijing, 30 September 2008

9

## Approaches to Concept Assignment

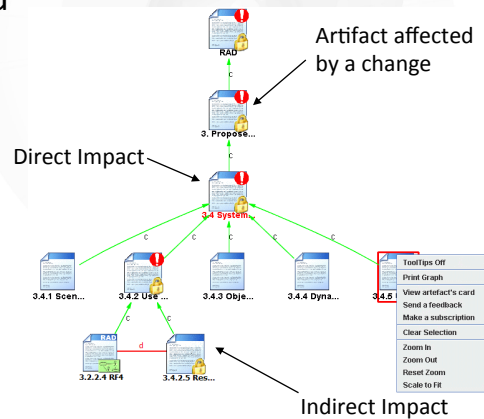
- Static analysis
- Dynamic analysis
- Use of informal information, e.g., the meaning of the program identifiers
- Natural language processing and Information Retrieval techniques
  - Identifiers in source code, documentation, CVS comments and change logs
- Combination of different approaches

Frontiers of Software Maintenance – Beijing, 30 September 2008

10

## Identifying the Candidate Impact Set

- Looking for artefacts that might be affected due to changes made to artefacts in the SIS
- Ripple effects may result in direct or indirect impact
- Use of dependencies among software artefacts and reachability analysis



Frontiers of Software Maintenance – Beijing, 30 September 2008

11

## Problems

- An under-estimate of the starting impact set might reduce the recall of the impact analysis process.
  - Affected components that are not reachable from the identified SIS are not obvious or easy to detect
- Reachability analysis performed without using any pruning strategy might result in a low precision
  - Not all reachable components are actually affected
  - Approaches to reducing the number of false positives
    - Considering the distance between artefacts
    - Associating traceability links with specific semantics or probabilities indicating the likelihood of an indirect impact
    - Defining propagation rules and using such rules to navigate the reachability graph
    - Incremental analysis of the traceability graph

Frontiers of Software Maintenance – Beijing, 30 September 2008

12

## Outline

- Introduction
- Impact Analysis
- Traceability Dimensions
- Traceability Challenges
- Conclusion

Adapted from: Bianchi  
et al. (IWPC, 2000)

| Dimension | Categories |                 |
|-----------|------------|-----------------|
| $D_1$     | Vertical   | Horizontal      |
| $D_2$     | Structural | Knowledge based |
| $D_3$     | Implicit   | Explicit        |

Frontiers of Software Maintenance – Beijing, 30 September 2008

13

## Vertical vs Horizontal Traceability (1)

- Vertical traceability: the ability to trace dependent artefacts within a model
  - Most approaches focused on source code artefacts
    - Program slicing to identify dependencies among statements
    - Source code analysis and reverse engineering to identify dependencies among source code modules
  - Other approaches focused on software artefacts at higher abstraction levels, such requirements, or UML models
- Horizontal traceability: the ability to trace artefacts between different models
  - Requirements, design models, source code, test cases, defect reports, rationale elements

Frontiers of Software Maintenance – Beijing, 30 September 2008

14

## Vertical vs Horizontal Traceability (2)

Software Engineering Lab  
UNIVERSITY OF SALERNO

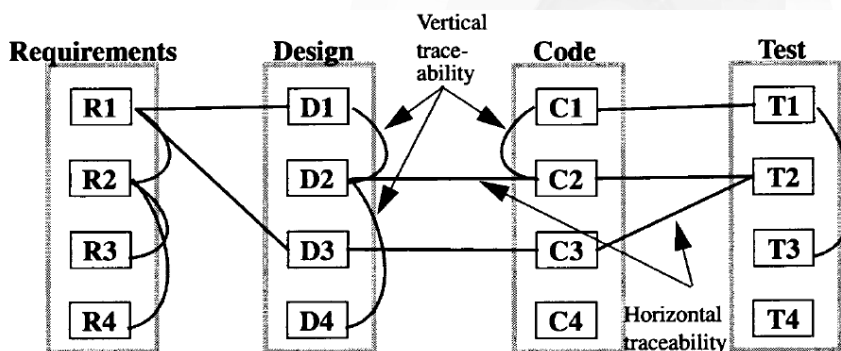


Figure taken from Lindvall and Sandahl (JSM 1998)  
Original classification by Pfleeger and Bohner (CSM 1990)

Frontiers of Software Maintenance – Beijing, 30 September 2008

15

## Structural vs Knowledge-based Traceability

Software Engineering Lab  
UNIVERSITY OF SALERNO

- Structural traceability: derived by parsing techniques
  - The artefacts are defined in a formal language
    - Not only source code, but also high level models expressed in a formal (e.g., XML) and/or graphical language (e.g., UML)
- Knowledge-based traceability: deals with informal information and requires human interaction
  - Heuristics + knowledge of the application/solution domain
    - Name tracing rules, dynamic analysis rules, logic based rules
    - Information retrieval and version history mining methods
  - Discovered links are validated to avoid false positives
  - Links may be missed (artefacts will not be included in CIS)

Frontiers of Software Maintenance – Beijing, 30 September 2008

16

## Implicit vs Explicit Traceability

- **Explicit Traceability:** Links are stored in some repository
  - Required when the recovery process is expensive or requires human interaction
- **Implicit Traceability:** Links are derived on-the-fly from software artefacts when needed
  - Used when the recovery process is fast and reliable
- **Hybrid approaches:** some type of links are stored explicitly while other types of links are recovered when needed

Frontiers of Software Maintenance – Beijing, 30 September 2008

17

## Outline

- Introduction
- Impact Analysis
- Traceability Dimensions
- Traceability Challenges
  - Traceability Recovery
  - Link evolution
- Conclusion

Frontiers of Software Maintenance – Beijing, 30 September 2008

18

## Traceability recovery

- Recovering knowledge-based traceability links
  - Heuristic based approaches
    - Name tracing: regular expressions exploiting naming conventions, string edit distance
    - Syntactic matches between requirements and object models
    - Dynamic analysis: identifying source code components involved during the execution of scenarios
  - Data mining-based approaches
    - Use of software configuration management repository history (e.g. CVS) to detect co-changes (logical coupling) between classes, files, and functions
    - Source code components changing together in the past are likely to affect each other again in case of future changes

Frontiers of Software Maintenance – Beijing, 30 September 2008

19

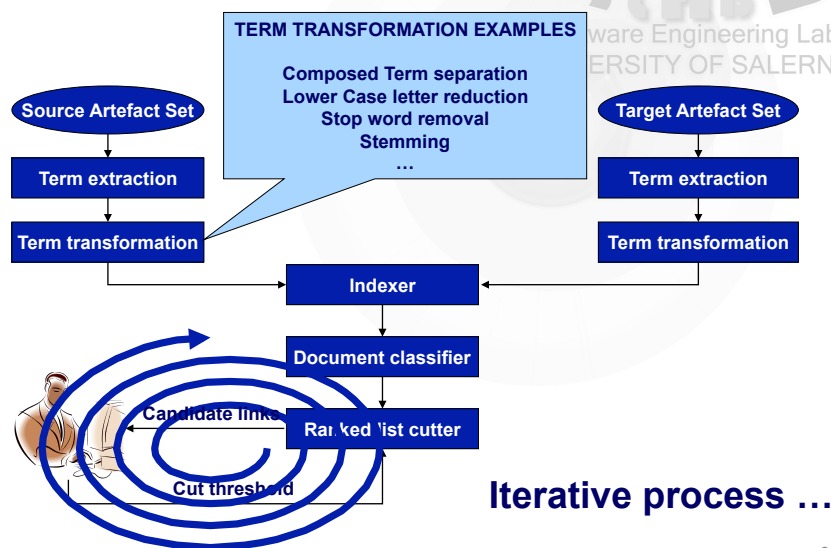
## Traceability recovery based on Information Retrieval methods

- Rationale
  - Source code artefacts contain text
    - Free text descriptions, comments, and identifiers
  - High textual similarity between two artefacts likely mirrors a traceability link
- Different methods
  - Vector Space Model, Probabilistic Model, LSI
  - Combination and enhancement of the basic methods
    - Thesaurus, keywords, ontology
    - Relevant feedbacks

Frontiers of Software Maintenance – Beijing, 30 September 2008

20

## IR-based traceability recovery process



Frontiers of Software Maintenance – Beijing, 30 September 2008

21

## Pro and Con of IR methods

- Pro: high density of correct links in the upper part of the ranked list
  - Low effort to classify correct links and discard false positives in the upper part of the list
- Con: correct links are also in the bottom part of the ranked list where the density of correct links is lower
  - The ranking does not help anymore
  - The effort to discard false positives is much higher than the effort to recover more correct links
  - Recovering all correct links requires analysing almost all possible links

Frontiers of Software Maintenance – Beijing, 30 September 2008

22

## Effectively using IR methods

- Using an Incremental approach
  - Starting with a high threshold
  - Classifying the candidate links (correct or false positives)
  - Decreasing the threshold and repeating the process
    - Until the effort to discard the false positives becomes much higher than the effort to validate correct links
- Improving the performances of IR methods
  - Improving text similarity among related artefacts
    - For example, IR methods can be used during programming to measure the similarity of source code classes with related high level artefacts
  - Combining IR methods with other approaches (e.g., version mining, or syntactic approaches)

Frontiers of Software Maintenance – Beijing, 30 September 2008

23

## Outline

- Introduction
- Impact Analysis
- Traceability Dimensions
- Traceability Challenges
  - Traceability Recovery
  - Link evolution
- Conclusion

Frontiers of Software Maintenance – Beijing, 30 September 2008

24

## Link Evolution

- Software evolve, so (explicit) traceability links may become out of date
  - The traceability matrix could become incomplete (missing new correct links) or might include no longer valid links
  - Maintaining traceability information up-to-date is a tedious and time consuming task
    - It is often sacrificed under the time pressure of on-going work
- Mainly a problem for knowledge-based links
  - Approaches to support the evolution of links between architectural and source code components
  - Analysis of textual similarity between traced artefacts
    - Looking for cases of low similarity ...

Frontiers of Software Maintenance – Beijing, 30 September 2008

25

## Outline

- Introduction
- Impact Analysis
- Traceability Dimensions
- Traceability Challenges
- Conclusion

Frontiers of Software Maintenance – Beijing, 30 September 2008

26

## Conclusion

- We addressed the support provided by traceability management during the software change impact analysis
- Traceability information are mainly useful during the identification of the Candidate Impact Set
- Traceability approaches can be classified according to different dimensions
- Main traceability challenges (i.e., recovery and evolution) involve knowledge-based traceability

Frontiers of Software Maintenance – Beijing, 30 September 2008

27

**Thank you!**

## Questions and/or comments

This work is supported by the project METAMORPHOS (MEthods and Tools for migrAting software systeMs towards web and service Oriented aRchitectures: exPerimental evaluation, usability, and tecHnOlogy tranSfer), funded by Ministero dell' Università e della Ricerca under grant PRIN-2006-2006098097.

Frontiers of Software Maintenance – Beijing, 30 September 2008

28